

“Recipe Sharing Platform with Meal Planner”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

THODETI ARUN 2520030456

PAVAN 2520090200

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Recipe Sharing Platform with Meal Planner**” is a record of bonafide work of Thodeti Arun (2520030456) and Pavan (2520090200), submitted in partial fulfillment of the requirements of the course Database Systems Engineering and Distributed Backend Development (25CS1302E), Section 10, Team 28, during 2026-2027, Trimester 4, at K L (Deemed to be) University. The results embodied in this report have not been copied from any other student, department, or institution.

THODETI ARUN – 2520030456

PAVAN – 25200902000

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled “**Recipe Sharing Platform with Meal Planner**” is a bonafide work done and submitted by **Thodeti Arun (2520030456)** and **Pavan (2520090200)** in partial fulfillment of the requirements of the **course Database Systems Engineering and Distributed Backend Development (25CS1302E)** at K L (Deemed to be) University during 2026-2027, **Trimester 4**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We also thank all teaching and non-teaching staff who helped us directly or indirectly, and our parents, friends and classmates for their cooperation during this work.

THODETI ARUN – 2520030456

PAVAN – 2520090200

Abstract

The Recipe Sharing Platform with Meal Planner is a web-based application that gives users one place to discover, share and manage recipes. Many people keep recipes in scattered places such as notebooks, chat messages and bookmarks, spend time repeating weekly meal planning, and find it hard to work out the combined ingredient list for several recipes. This project solves these problems with an integrated platform where users create accounts, publish recipes, browse recipes shared by others, and search by category.

Users record each recipe's ingredients, cooking instructions, preparation time, category and images. They can rate recipes and save them for later. A weekly meal planner lets them arrange chosen recipes across the days of a week, and the system then generates a consolidated grocery list from the ingredients of the planned recipes. Nutrition information, fetched from an external Nutrition API, helps users understand the nutritional value of their meals. The application uses a React.js frontend, a Node.js and Express.js backend, a MongoDB database, and Cloudinary for image storage. The database holds users, recipes, meal plans and grocery lists, with relationships maintained through document references and integrity rules enforced through schema validation and unique indexes. The platform aims to simplify recipe management, meal planning and grocery preparation.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Cooking at home involves three recurring tasks: finding recipes, deciding what to cook across the week, and buying the right ingredients. Today these are handled with separate tools or none at all. Recipes live in screenshots, video bookmarks and handwritten notes; meal plans are made from memory; and shopping lists are written by hand after reading every recipe one by one. This is slow, and it is easy to forget an ingredient or buy duplicates.

The Recipe Sharing Platform with Meal Planner brings these tasks together in one database-backed web application. Recipes, meal plans and grocery lists are all stored centrally, so a recipe that a user finds today can be saved, placed in next week's plan, and turned into a shopping list without re-typing anything.

1.2 Problem Statement

Recipes are saved in scattered places, weekly meal planning takes repeated effort, and preparing a combined ingredient list across several recipes is difficult. There is a need for one simple application that lets users store and share recipes, organize them into a weekly plan, and obtain the ingredients they need to buy.

1.3 Scope of the Project

The project covers: user registration and login; creation, editing, deletion and browsing of recipes with ingredients, instructions, category, preparation time and images; search and filtering by category; ratings and saved recipes; a weekly meal planner; automatic grocery list generation; and nutrition information retrieved from an external API. Payments, grocery delivery integration, social networking features (followers, comments) and a native mobile application are outside the scope of this submission.

1.4 Objectives

- Provide user accounts for creating, browsing and sharing recipes.
- Support recipe search by category and by recipe details.
- Organize selected recipes into a weekly meal plan.
- Generate a grocery list from the ingredients of the planned recipes.
- Show nutrition information when it is available.

- Let users rate recipes and save them for future use.
- Protect user data with hashed passwords and token-based access control.

1.5 Importance of the Application

By keeping recipes, plans and ingredient lists in one place, the application saves users the time spent on planning and shopping and reduces food waste caused by forgotten or duplicated purchases. Nutrition information supports more informed food choices.

1.6 Target Users / Audience

The system is intended for registered users who cook at home, including students, working professionals and families. Every registered user can publish and browse recipes, rate and save them, build a weekly meal plan, and generate a grocery list. Visitors who are not logged in can browse and search recipes but cannot rate, save or plan.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight. A development or demonstration machine with a dual-core processor, 4 GB of RAM and about 500 MB of free disk space is sufficient to run the frontend, the backend and a local MongoDB instance together. Images are stored on Cloudinary, so they do not use local disk space.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MongoDB (local Community Server or a MongoDB Atlas cluster)
- A modern web browser (Chrome, Edge or Firefox)
- Git, for version control
- A Cloudinary account (free tier) for image storage
- An API key for the chosen Nutrition API
- Visual Studio Code and MongoDB Compass (recommended for development)

2.3 Development Tools and Frameworks

React.js is used for the frontend, Node.js with Express.js for the backend REST API, and MongoDB for storage, accessed through the Mongoose library. Postman is used for API testing, Git and GitHub for source control, and Visual Studio Code is the main development environment. The programming language across the whole stack is JavaScript.

CHAPTER 3: TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	React.js	Single-page user interface
Backend	Node.js, Express.js	REST API and business logic
Database	MongoDB (Mongoose ODM)	Storage of users, recipes, meal plans, grocery lists
Image storage	Cloudinary	Hosting and delivery of recipe images
External API	Nutrition API	Nutritional values for recipes
Testing	Postman	API testing
Version control	Git and GitHub	Source control and collaboration
Language / IDE	JavaScript, Visual Studio Code	Development

3.1 Frontend

The frontend is a React.js single-page application. It has pages for the home screen, login and registration, recipe browsing and details, the recipe form, and a dashboard containing saved recipes, the meal plan and the grocery list. All data is obtained by calling the backend REST API over HTTP; the frontend never connects to the database directly.

3.2 Backend

The backend is a REST API built with Node.js and Express.js. It is organized into routes, controllers and models: routes declare URLs and required authentication, controllers validate requests and apply business rules, and models (Mongoose schemas) define and access the data. This separation keeps each file focused on one responsibility.

3.3 Database

MongoDB stores data as flexible JSON-like documents, which suits recipes well because each recipe naturally contains nested lists of ingredients and steps. Four collections are used: Users, Recipes, MealPlans and GroceryLists. Mongoose schemas add required fields, data types, allowed values and unique indexes on top of MongoDB's flexibility.

3.4 Version Control

Git is used throughout development, with one repository containing the client and server code, hosted on GitHub.

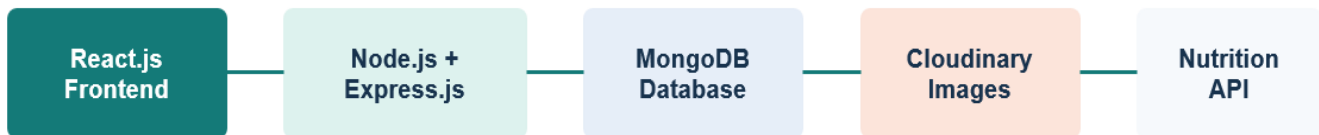
3.5 Authentication and Security Libraries

Passwords are hashed with bcrypt before they are stored, so plain-text passwords are never written to the database. Login sessions use stateless JSON Web Tokens (JWT) signed with a server-side secret. Express middleware verifies the token on every protected route, and secrets (database URI, JWT secret, API keys) are kept in environment variables rather than in source code.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a three-tier architecture with two external services attached to the backend.



4.2 Description of Each Layer

Presentation layer. The React.js application renders pages and forms, keeps the logged-in user's token, and sends every request to the backend API. It validates input on the client for a good user experience, but the server remains the authority.

Application layer. The Express.js API receives requests, passes protected ones through authentication middleware that verifies the JWT and identifies the user, and then hands them to controllers. Controllers validate the data, apply rules (for example, only a recipe's author may edit or delete it) and use models to read and write data. The backend is also the only component that talks to Cloudinary and the Nutrition API.

Data layer. MongoDB stores all persistent data. Cloudinary stores the image files themselves, while MongoDB stores only their URLs.

4.3 Deployment Architecture

During development, the React development server, the Express server and MongoDB run as separate local processes on one machine. For a hosted demonstration, the recommended arrangement is the frontend on a static hosting service, the backend on a Node.js hosting service, and the database on MongoDB Atlas. Chapter 9 describes the steps.

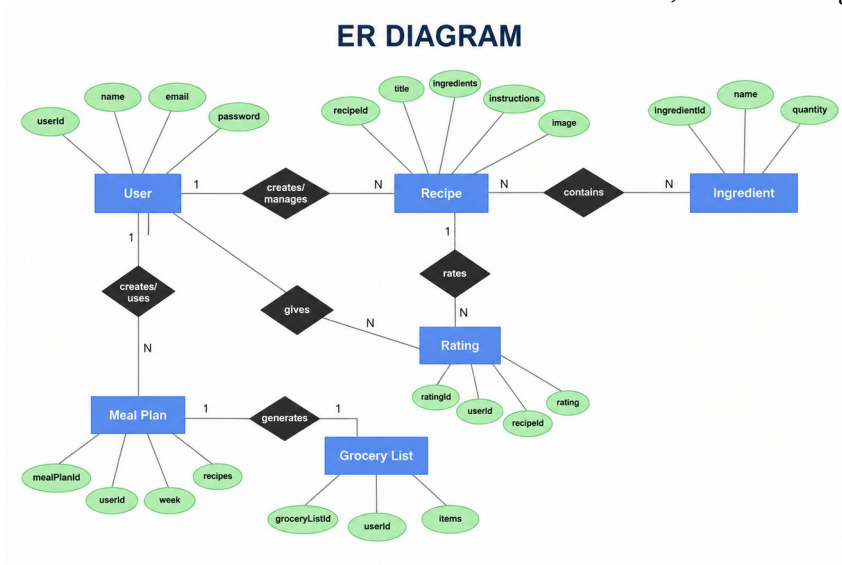
CHAPTER 5: DESIGN

5.1 Database Design Principles

The data model follows how the application reads data. Data that is always read together is embedded in one document (a recipe's ingredients, steps and ratings); data that is shared or grows independently is stored in separate collections and linked by reference (a recipe's author, the recipes inside a meal plan). Integrity is enforced at the database layer through required fields, enumerated values, range limits and unique indexes, so invalid data is rejected even if application code has a bug.

5.2 ER Diagram / Database Schema

The database consists of seven related tables, shown in Figure 5.1.



Sample records

```
User      { name: Arun, email: arun@example.com }
Recipe     { title: Vegetable Pasta, category: Dinner, prepTime: 25,
servings: 2 }
MealPlan   { week: 1, entries: [ { day: Monday, mealType: Dinner, recipe:
Vegetable Pasta } ] }
```

5.3 Design Decisions and Assumptions

- **Embedding versus referencing.** Ingredients, instructions and ratings are embedded in the recipe because they are always loaded with it and have no meaning on their own. Meal plans reference recipes instead of copying them, so edits to a recipe are reflected in every plan that uses it.

- **Ratings inside the recipe.** Each recipe stores its ratings and a precomputed average, so listing recipes with star ratings needs no extra query. A rating is identified by user, so a user can only update their own rating rather than add a second one.
- **Nutrition data is cached.** Nutrition values are fetched once when a recipe is created or edited and stored in the recipe, which avoids repeated calls to the external API and lets the recipe still display if the API is unavailable.
- **Grocery list merging.** Ingredients from all planned recipes are combined by matching the ingredient name (case-insensitive) and unit and adding quantities. The same item in different units stays on separate lines; unit conversion is a future enhancement.
- **Images outside the database.** Only Cloudinary URLs are stored.
- **Who may modify what.** Anyone may read recipes; only the author may update or delete a recipe; meal plans, grocery lists and saved recipes are visible only to their owner.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

- The backend is divided into modules, each with its own routes, controller and model: Authentication, Recipes, Meal Plans and Grocery Lists, plus service modules for Cloudinary and the Nutrition API.
- **Authentication module.** Registration validates the input, checks that the email is not already used, hashes the password with bcrypt and stores the user. Login compares the submitted password with the stored hash and, if it matches, returns a signed JWT.
- **Recipe module.** Supports creating, reading, updating and deleting recipes; searching by title text and filtering by category with pagination; rating; and saving or un-saving a recipe. When a recipe is created, the controller uploads the image to Cloudinary and requests nutrition data before saving the document.
- **Meal plan module.** Creates or updates the plan for a given week by replacing its entries, and returns the plan with the referenced recipes populated so the frontend can display titles and images.
- **Grocery list module.** The generation routine reads all recipes referenced by a meal plan, collects their ingredients, merges duplicates by (name, unit) while adding quantities, and stores the result as a grocery list. Users can tick items off as they buy them.

6.2 Front-end Logic

The frontend uses React Router for navigation between the public pages (home, recipe search, login, registration) and the pages that need login (recipe form, dashboard). The logged-in user and token are kept in a React Context so every component can check whether the user is signed in. Other state stays local to the component that needs it, which avoids the extra complexity of a state-management library at this project's scale. Reusable components such as the recipe card, rating stars, form inputs and loading or error messages are written once and reused across pages.

6.3 API Integration

All HTTP calls go through one central Axios instance that adds the JWT to outgoing requests and sends the user back to the login page if the server responds with 401 Unauthorized. Each backend resource has a matching frontend service file (for example `recipeService` and `mealPlanService`), so pages never hard-code URLs. On the backend, the Cloudinary and Nutrition API calls are isolated in their own service files so that they can be replaced or mocked without touching the controllers.

6.4 Authentication and Authorization

A `protect` middleware verifies the JWT on each protected route and attaches the user's identity to the request. Ownership checks are then applied in the controllers: a user can edit or delete only their own recipes, and can read only their own meal plans, grocery lists and saved recipes. Responses never include the password hash.

CHAPTER 7: FEATURES

7.1 List of Main Features

- User registration and login with secure password storage
- Create, edit and delete recipes with ingredients, instructions, preparation time, category and image
- Browse recipes, search by title and filter by category
- View full recipe details, including author, ratings and nutrition information
- Rate recipes from 1 to 5 stars
- Save recipes and view them on the dashboard
- Weekly meal planner with day and meal-type slots
- Automatic grocery list generation from the meal plan, with check-off for purchased items
- Nutrition information through an external API

7.2 How Each Feature Works

- **Recipe search.** The user types a keyword and/or selects a category. The backend runs a text search on recipe titles (and descriptions), applies the category filter, sorts the results and returns one page at a time.
- **Ratings.** A logged-in user picks 1-5 stars. The backend stores or updates that user's rating on the recipe and recalculates the average rating.
- **Meal planner.** The user chooses a week, then assigns recipes from saved or browsed recipes to day and meal-type slots. The plan is stored as a list of entries referencing the recipes.
- **Grocery list generation.** When the user presses "Generate grocery list", the backend gathers the ingredients of every recipe in that week's plan and adds up duplicates. For example, if two planned recipes each need 200 g of tomatoes, the list shows one line with 400 g of tomatoes.
- **Nutrition information.** When a recipe is saved, the backend sends its ingredient list to the Nutrition API and stores the calories, protein, carbohydrates and fat it returns. If the API cannot provide a value, the recipe page shows that nutrition information is not available.

CHAPTER 8: TESTING

8.1 Postman API Testing

A Postman collection will be built for all endpoints, organized into folders (Auth, Recipes, Meal Plans, Grocery Lists). The Login request will carry a small script that saves the returned JWT into a collection variable so that later requests use it automatically, and requests such as Create Recipe, Add to Meal Plan and Generate Grocery List will share variables so a full workflow can run without copying IDs by hand.

8.2 Integration Testing

Each backend module is to be tested against a real running MongoDB instance before the next module is built on it, rather than against mocked data. As a final check, the database will be emptied and the complete workflow (register, create recipe, rate, save, plan a week, generate grocery list) re-run from a clean start.

8.3 Tools Used for Testing

- Postman - manual and scripted API testing
- MongoDB Compass or mongosh - direct inspection of stored documents and indexes
- Browser developer tools - checking network calls and frontend behaviour
- Manual browser testing - checking pages, forms and navigation

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Status
1	Register with an email that already exists	409 Conflict, no duplicate user created	To be run
2	Login with a wrong password	401 Unauthorized	To be run
3	Open a protected route without a token	401 Unauthorized	To be run
4	Check a user document in the database	Password stored as a bcrypt hash, not plain text	To be run
5	Create a recipe without a title or ingredients	400 Bad Request with a validation message	To be run

#	Test Case	Expected Result	Status
6	Edit or delete another user's recipe	403 Forbidden	To be run
7	Search by keyword and filter by category	Only matching recipes returned, paginated	To be run
8	Rate a recipe twice as the same user	Existing rating updated; average recalculated; no second rating stored	To be run
9	Submit a rating of 6	400 Bad Request	To be run
10	Save and then un-save a recipe	Recipe added to and removed from the user's saved list	To be run
11	Upload a recipe image	Image stored on Cloudinary; URL saved with recipe	To be run
12	Create a meal plan twice for the same week	Existing plan updated; no duplicate plan for the week	To be run
13	Generate a grocery list from two recipes sharing an ingredient	Duplicate ingredients merged with quantities added	To be run
14	Request nutrition data when the external API is unavailable	Recipe still saved; nutrition shown as not available	To be run
15	View another user's meal plan or grocery list	403 Forbidden or 404 Not Found	To be run

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

Running locally for development and demonstration:

1. Clone the repository and start MongoDB (local instance or a MongoDB Atlas connection string).
2. Create the `.env` file in the `server` folder (see 9.2).
3. Start the backend: `cd server, npm install, npm run dev`.
4. Start the frontend: `cd client, npm install, npm start` (or `npm run dev`).
5. Open the URL shown by the frontend development server in a browser and register a new account.

9.2 Environment Configuration

All configuration is kept in a `.env` file that is never committed to version control. It contains the MongoDB connection string (`MONGO_URI`), the JWT signing secret and expiry (`JWT_SECRET`, `JWT_EXPIRES_IN`), the Cloudinary credentials (`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`), the Nutrition API key (`NUTRITION_API_KEY`), the server port (`PORT`) and the frontend URL for CORS (`CLIENT_URL`). A `.env.example` file with placeholder values is committed so the required variables are documented without exposing real credentials.

9.3 Hosting of Frontend and Backend

- 10 For this submission the application runs locally. If a hosted demonstration is needed, the suggested approach is: database on MongoDB Atlas (free tier), backend on a Node.js hosting service such as Render, and the built React frontend on a static host such as Netlify or Vercel, with the environment variables above configured in each service's settings and the backend's CORS setting pointed at the frontend's public URL.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Embedding versus referencing in MongoDB.** MongoDB has no joins, so it was unclear whether ratings, ingredients and meal-plan recipes should be embedded or stored in separate collections. Embedding everything risks duplicated data and documents that grow without limit; referencing everything makes simple reads slow and complicated.
- **Inconsistent ingredient entries.** Users can write the same ingredient in different ways (“Tomato”, “tomatoes”, “200 g”, “0.2 kg”), which makes it hard to combine ingredients into one grocery list.
- **Dependence on an external Nutrition API.** The API is not yet finalized, and any choice brings request limits and incomplete data for some ingredients. The recipe feature must not fail when the API cannot answer.
- **Image storage.** Storing image files in MongoDB would make the database large and slow, so a separate image service was needed.

10.2 Solutions Applied

Challenge	Planned Solution
Combining ingredients written in different ways (for example “Tomato” and “tomatoes”, or different units)	Normalize names to lower case, offer a fixed list of units in the recipe form, and merge only when name and unit match
Dependence on an external Nutrition API (rate limits, missing data)	Call the API only from the backend, store the result in the recipe, and show “not available” when data is missing
Keeping the database small with image-heavy content	Store images on Cloudinary and keep only the URL in MongoDB
Protecting user data and API keys	bcrypt password hashing, JWT authentication, ownership checks and environment variables for secrets
Keeping average ratings correct	Store one rating per user and recalculate the average on every change

10.3 Current Limitations or Known Bugs

- No email verification or password-reset flow.
- Grocery list merging does not convert between different units.
- Nutrition values depend on the external API’s coverage and are approximate.
- Tested only on a local deployment; not yet hosted publicly.

- Image upload size and format are limited by the Cloudinary free tier.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Email-based account verification and password reset.
- Unit conversion in the grocery list (for example grams to kilograms).
- Comments on recipes and the ability to follow other users.
- Recipe collections or folders (for example “Weeknight dinners”).
- Nutrition totals per day and per week in the meal planner.
- Printable and shareable grocery lists.

11.2 Possible Integrations or Optimizations

- Caching frequently requested recipe searches to reduce database load.
- Integration with grocery delivery services.
- Containerizing the application with Docker.
- An automated test suite (Jest and Supertest) to complement manual Postman testing.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project aims to replace scattered recipe storage and manual meal planning with a single web application where users can share recipes, plan a week of meals and obtain a ready grocery list, with nutrition information alongside. The design uses a React.js frontend, an Express.js REST API, a MongoDB document database, Cloudinary for images and an external Nutrition API.

12.2 What Was Achieved

The problem has been defined, objectives and scope set, the architecture designed, the technology stack chosen and the database collections, fields and relationships specified. The implementation and testing described in this report are the remaining work, to be completed and evidenced for the final submission.

12.3 Skills Learned

The project provides practical experience in document-oriented database design (embedding versus referencing, indexes, validation), designing REST APIs with meaningful status codes, token-based authentication, integrating third-party services (Cloudinary and a Nutrition API), building a React frontend that consumes an API, and systematic API testing with Postman.

CHAPTER 13: REFERENCES

- React - Official Documentation, <https://react.dev>
- Node.js - Official Documentation, <https://nodejs.org/en/docs>
- Express.js - Official Documentation, <https://expressjs.com>
- MongoDB Manual, <https://www.mongodb.com/docs/manual/>
- Mongoose - Official Documentation, <https://mongoosejs.com/docs/>
- Cloudinary Documentation, <https://cloudinary.com/documentation>
- jsonwebtoken and bcrypt - npm documentation, <https://www.npmjs.com>
- Postman Learning Center, <https://learning.postman.com>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core idea used when generating a grocery list (proposed implementation):

// Merge ingredients from all recipes in a meal plan

```
const merged = new Map();
for (const recipe of recipes) {
  for (const ing of recipe.ingredients) {
    const key = ing.name.trim().toLowerCase() + '|' + ing.unit;
    if (merged.has(key)) {
      merged.get(key).quantity += ing.quantity;
    } else {
      merged.set(key, { name: ing.name, quantity: ing.quantity, unit: ing.unit, checked: false });
    }
  }
}
const items = Array.from(merged.values());
```

Appendix B: Installation and Setup Instructions

- Install Node.js 18 or later and MongoDB (or create a free MongoDB Atlas cluster).
- Clone the repository: `git clone <repository-url>`
- In the server folder, copy `.env.example` to `.env` and fill in `MONGO_URI`, `JWT_SECRET`, the Cloudinary keys, `NUTRITION_API_KEY`, `PORT` and `CLIENT_URL`.
- Start the backend: `cd server && npm install && npm run dev`
- Start the frontend: `cd client && npm install && npm start`
- Open the address shown by the frontend (for example `http://localhost:3000`) in a browser.

Appendix C: Default Login Credentials

The application has a single user role, so there are no separate admin accounts. The accounts below are the planned demo users to be created by a seed script; confirm or change them once the seed script exists. Use these for demonstration only.

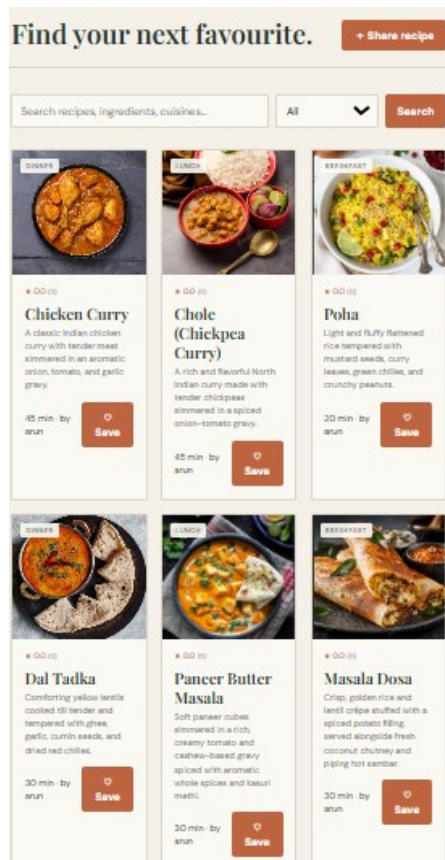
R	Role	Email	Password
	User	A arunarun888t@gmail.com	ArunArun@456
	User	Papavan1564@gmail.com	Pavan@123

Appendix D: API Reference Summary

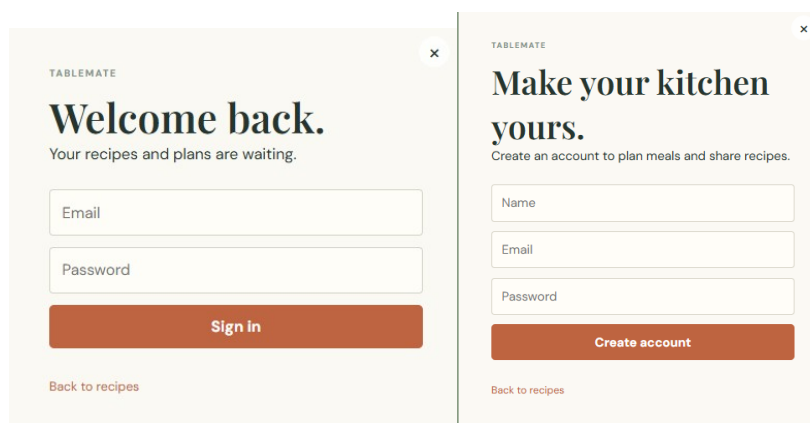
Resource	Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Recipes	GET /api/recipes (search, category, page), GET /api/recipes/:id, POST /api/recipes, PUT /api/recipes/:id, DELETE /api/recipes/:id, GET /api/recipes/mine
Ratings and saving	POST /api/recipes/:id/rate, POST /api/recipes/:id/save, GET /api/recipes/saved
Meal plans	GET /api/mealplans/:weekStart, PUT /api/mealplans/:weekStart
Grocery lists	POST /api/grocery/generate, GET /api/grocery/:mealPlanId, PATCH /api/grocery/:id/items/:itemId

Appendix E: Screenshots

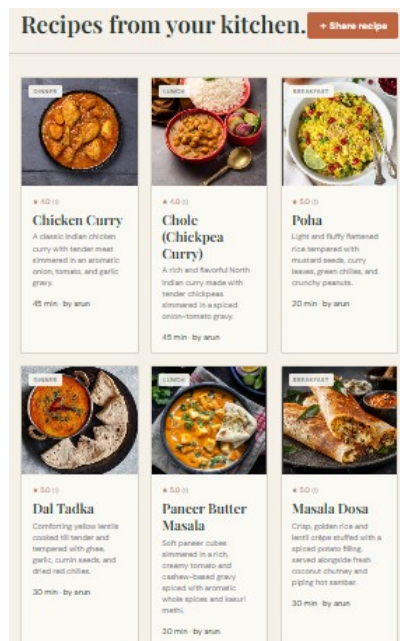
1. Home page (search, categories, featured recipes)



2. Login and registration pages



3. Recipe listing and recipe details page



4. Add recipe form (ingredients, instructions, image upload)

Share a recipe with the community. [+ Share recipe](#)

Title: Category:

Description:

Prep time: Cook time:

Image URL:

Ingredients:

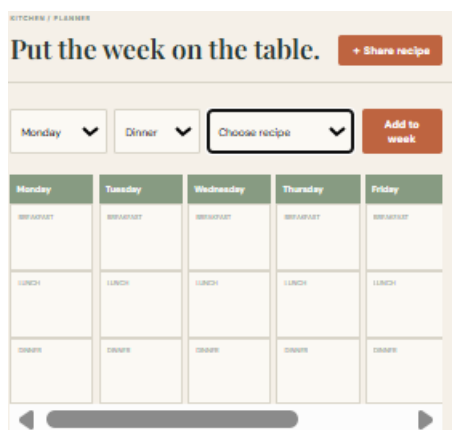
Instructions:

[Publish recipe](#)

5. Dashboard: saved recipes



6. Dashboard: weekly meal planner



7. Dashboard: grocery list

